CS2201 – DATABASE MANAGEMENT

SYSTEMS Mini Project Report

Student Record Management System

Submitted

By

Ullas Gowda M (1RUA25BCA0112)

Varun R (1RUA25BCA0117)

Raghavendra N (1RUA25BCA0085)

Under the guidance of:

Dr./Prof. Pranjul Shukla

School of Computer Science and Engineering RV University, Bangalore

CERTIFICATE

Certified that the CS2201 – Database Management Systems, Project work titled Student Record Management System is carried out by **Ullas Gowda M(1RUA25BCA0112)**, **Varun R (1RUA25BCA0117)** and **Raghavendra N(1RUA25BCA0085)** who are Bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2026–27. It is certified that all corrections/ suggestions from all the continuous internal evaluations have been incorporated into the project and in this report.

Dr./ Prof. Pranjul Shukla

Faculty Guide Program Director

Abstract

This project presents a Student Record Management System, a small browser-based web application that lets users manage student information without entering every detail directly into MySQL. The problem addressed is the difficulty of handling student records through raw database commands, which is slow, error-prone, and inconvenient for everyday tasks. The objectives are to provide a simple interface for adding, viewing, editing, deleting, and searching student records, to display useful summary statistics on a dashboard, and to demonstrate how a web application connects to a relational database. The system was developed using Python and Flask for the application logic and routing, MySQL for data storage, mysql-connector python for database communication, and HTML and CSS for the user interface. The methodology follows the CRUD (Create, Read, Update, Delete) model, with each operation implemented as a Flask route that runs SQL queries against a students table. The outcome is a working prototype that performs all core record-management tasks, generates student IDs automatically, and prevents duplicate email addresses. The project is a learning exercise rather than a fully secured production system.

1. Introduction

Educational institutions maintain student information such as student names, email addresses, courses and semester details. These records need to be stored accurately and should be easy to access, add, modify and remove. Managing every record by directly writing SQL commands can be inconvenient for users who are not familiar with database queries.

The Student Record Management System was developed as a desktop application using Java Swing and MySQL. It provides a graphical user interface containing text fields, buttons and a table. Users can add, view, update and delete student records through the application instead of writing SQL commands for every operation. Java Database Connectivity (JDBC) is used to connect the Java application to the MySQL database.

The project was developed using Apache NetBeans as the IDE, Java for application logic, Swing for the graphical interface, JDBC for database communication and MySQL for permanent data storage. It demonstrates the CRUD operations—Create, Read, Update and Delete—which are commonly used in database applications. The project provides practical experience in Java programming, GUI development and relational database management.

2. Scope of the Project

2.1 Boundaries

The system manages student records stored in a local MySQL database named college. Its students table contains student_id, name, email, course and semester. The Java Swing interface provides options to add a student, view saved records, update an existing record and delete a selected record. Student information is stored in MySQL and can also be checked using MySQL Workbench.

2.2 Assumptions

- Java Development Kit (JDK) and Apache NetBeans are installed on the computer. • MySQL Server and MySQL Workbench are installed, and the MySQL server is running locally.
- The user has permission to create and use the college database and students table. • The Maven project includes the MySQL Connector/J JDBC driver and uses the correct local database connection settings.

2.3 Limitations

- The system is a learning project and is not intended to be a fully secured production application. • The application is designed for a local MySQL installation and does not include online hosting or multi-user administration.
- The current version stores basic student details only and does not include advanced features such as login authentication, role-based access or detailed reporting.
- The database must be created in MySQL before the application can use it. If a students table already exists with a different structure, its columns should be checked and corrected if necessary.

2.4 Intended Applications

The system is intended for students learning Java programming, Swing GUI development, JDBC connectivity and database management. It can also serve as a starting point for small student-record applications in colleges, training centres and educational departments.

3. Coding

3.1 Project Structure

The project is a Maven-based Java application. The main source code is stored in the standard Maven source directory, while pom.xml manages the project configuration and external dependencies. The structure is shown below.

```
StudentManagementSystem/
├── pom.xml
├── src/
│   └── main/
│       └── java/
│           └── com/mycompany/studentmanagementsystem/
│               └── StudentManagementSystem.java
└── target/ (generated build output)
```

The main Java class is StudentManagementSystem.java. It contains the Swing interface, event-handling code and JDBC operations. The pom.xml file contains the Maven configuration and the MySQL Connector/J dependency.

The project was created and run in Apache NetBeans as a Maven project named StudentManagementSystem.

3.2 Database Design

The MySQL database college contains a table named students. Each row represents one student. The student_id column is the primary key and is generated automatically using AUTO_INCREMENT. The other columns store the student's name, email, course and semester.

3.3 Database Connection

The Java application connects to the local MySQL server using Java Database Connectivity (JDBC). The Maven project includes MySQL Connector/J version 9.5.0 as a dependency. The connection uses the database URL jdbc:mysql://localhost:3306/college and the local MySQL username. The password must match the password configured for the local MySQL account.

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/college",
    "root",
    "YOUR_MYSQL_PASSWORD"
);
```

The password above is a placeholder. Use the password configured on the development computer. Do not publish a real database password in a public GitHub repository.

)

3.4 Application Modules

The application is organised around Java Swing components and event-handling methods. When a user clicks a button, the corresponding Java method executes an SQL statement through JDBC to perform the requested operation on the MySQL database.

- Add Student: accepts the student's name, email, course and semester from the text fields and inserts the details into the students table using an INSERT statement. A success message is displayed after a successful insertion.
- View Students: retrieves saved records using a SELECT statement and displays them in a JTable in the application window.
- Update Student: updates the selected student's details in the database using an UPDATE statement.
- Delete Student: deletes the selected student record using a DELETE statement. The application also provides a Clear Fields option to clear the input fields.

3.5 Running the Application

To run the application, start the MySQL Server, open the StudentManagementSystem Maven project in Apache NetBeans, confirm that the MySQL Connector/J dependency is available, and check the JDBC connection settings. Then select Run Project in NetBeans. The Java Swing window opens and the user can perform the student record operations. MySQL Workbench can be used to verify the stored records.

3.6 Observations and Challenges

During development, MySQL Server and MySQL Workbench were configured, and the Maven project was connected to MySQL by adding the MySQL Connector/J dependency. The Java package and main-class configuration also had to match the Maven project structure. After the setup was corrected, the Swing application opened successfully.

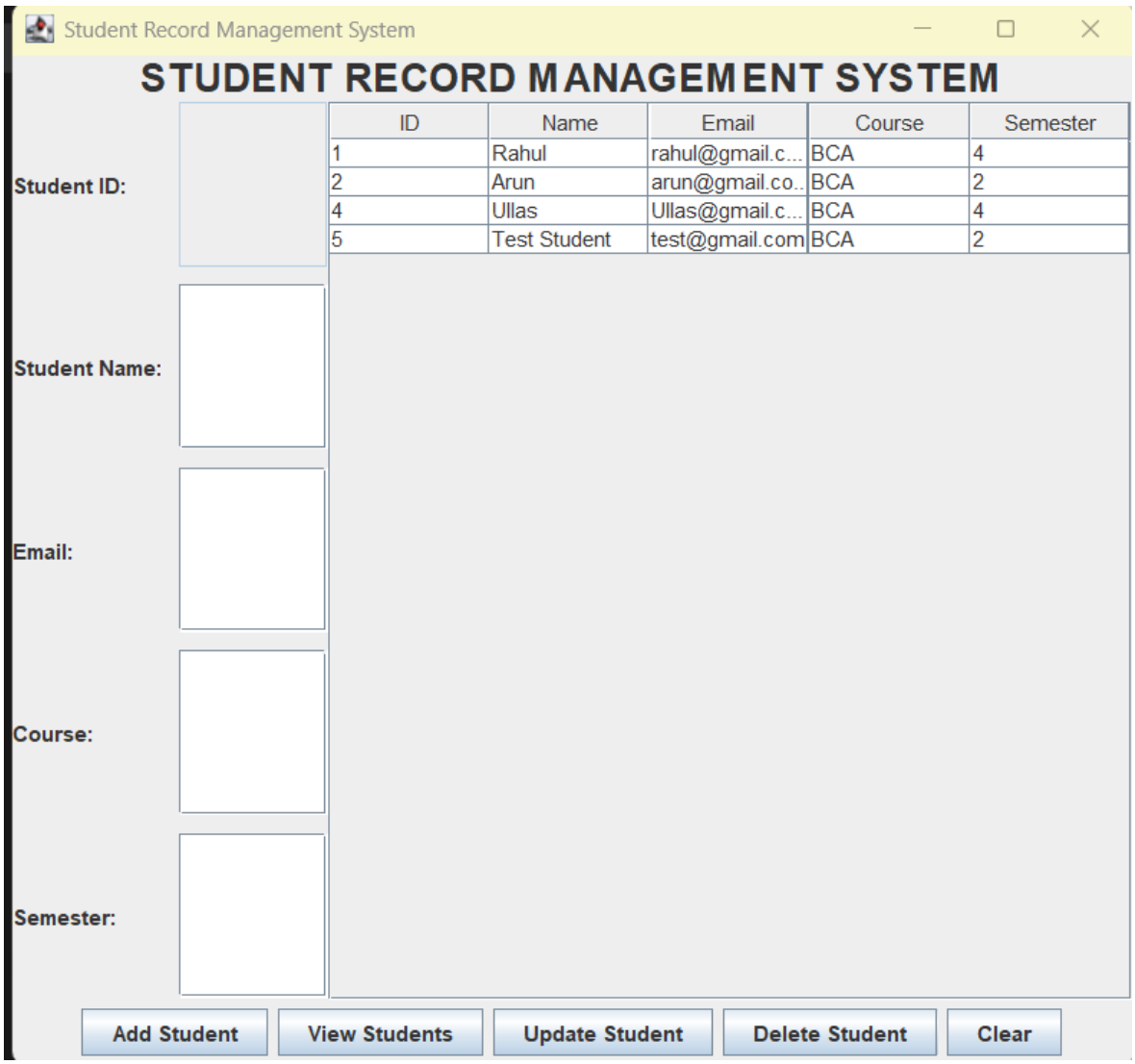
The Add Student operation was tested by entering a sample student record through the GUI. A success message appeared, and the record was verified in MySQL Workbench by running USE college; followed by SELECT * FROM students;. The View Students operation was checked, and the Update and Delete operations were also tested successfully.

3.7 Effectiveness of the Solution

The solution meets its main objective of providing a simple desktop interface for managing records stored in an actual MySQL database. Add, View, Update and Delete operations were tested. The successful insertion of a record through the Java application and its verification in MySQL Workbench confirm that the application communicates with the database through

JDBC rather than storing data only in temporary variables.

4. Screenshots



5. Conclusion

The project was developed to make student record management easier through a desktop graphical interface. Java Swing provides the window, labels, text fields, buttons and table, while JDBC connects the application to MySQL. The college database stores student details in the students table. The application supports the main CRUD operations: adding, viewing, updating and deleting student records.

During development, the project demonstrated how the different technologies work together. The user interacts with the Swing interface, Java event-handling methods process button clicks, JDBC sends SQL statements to MySQL, and MySQL stores or retrieves the records. Testing confirmed that a student added through the application appeared in the MySQL students table. The View, Update and Delete operations were also tested successfully.

The project provided practical experience with Java programming, Swing components, JDBC, Maven dependencies, SQL statements and MySQL Workbench. It can be improved in the future by adding search functionality, input validation, user authentication, reports and a more

advanced interface. Database credentials should be stored securely and must not be exposed in a public repository.

6. AI Tool Usage Report

6.1 Name of the AI Tool Used

ChatGPT.

6.2 Purpose of Usage

ChatGPT was used as a learning and reference tool to understand Java Swing, JDBC connectivity, MySQL database operations and the use of Maven dependencies. It also provided step-by-step guidance for configuring the MySQL connection, resolving NetBeans errors and understanding DBMS concepts relevant to the project.

The project code was developed and tested in Apache NetBeans. ChatGPT was used for explanations, code guidance and troubleshooting. The application itself is a Java Swing desktop application connected to MySQL; it does not use Flask, Python, HTML or CSS.

6.3 Sample Prompts Used

- “Help me create a Student Record Management System using Java and MySQL.”
- “How do I connect my Java application to MySQL using JDBC?”
- “Explain Java Swing and its purpose, and help me troubleshoot errors in NetBeans.”

6.4 Generated Outputs

- The Java Swing application uses JDBC to connect to the local MySQL database named college and the students table.
- The user enters student details in the Swing text fields and clicks Add Student.
- Java sends an INSERT statement through JDBC to store the record in MySQL.
- The View Students operation retrieves saved records with a SELECT statement.
- The records are displayed in the application's JTable.
  - Changes made through Update Student and Delete Student are applied to the MySQL table.

• The database contents can be independently checked in MySQL Workbench.

Example

The database records can be checked in MySQL Workbench with the following SQL statements:


```
USE college;



SELECT *

FROM students;
```

VALUES

The sample student record used during testing was added through the Java application and then verified in the MySQL students table.

The Swing application retrieves student records from MySQL when the View Students operation is performed. The information displayed in the application therefore comes from the database.

The same database is used for all operations: when a record is added, updated or deleted through the Java application, the corresponding change is made in MySQL.

The same works in reverse: **if you add a student through your Flask website, the record will be saved in MySQL too.**"

Java Swing is used to build the graphical interface. Components such as JFrame, JButton, JLabel, JTextField and JTable allow users to interact with the application through a window, buttons, text fields and a table.

JDBC (Java Database Connectivity) is the API that allows Java programs to connect to databases, send SQL statements and process query results. MySQL Connector/J is the driver used by this Maven project.

The Maven project configuration is stored in the pom.xml file.

The MySQL Connector/J dependency was added to pom.xml so that the Java application could communicate with MySQL.

Maven downloads and manages the required dependency for the project.

The Java source file contains the application logic and the Swing interface.

The application uses SQL INSERT, SELECT, UPDATE and DELETE statements

to manage student records.

The database is created and checked using MySQL Workbench.

The application was run from Apache NetBeans after the MySQL service and connection settings were checked.

The successful database tests confirmed that the Java application could store and retrieve student information through JDBC.

6.5 Modifications Made by the Student

The application was adapted and tested to match the Student Record Management System requirements. The interface contains fields for student name, email, course and semester, along with buttons for adding, viewing, updating and deleting records. A success message was displayed after adding a student. The record was checked in MySQL Workbench, and the View, Update and Delete operations were tested successfully.

6.6 Contribution of the AI Tool to the Project

ChatGPT helped with understanding Java Swing, JDBC connectivity, Maven configuration and MySQL queries, and with interpreting setup and runtime errors. The project was implemented and tested in Apache NetBeans with a real MySQL database. The student should ensure that the final screenshots, source code and explanation accurately represent the work completed by the project team.